

MedAssist AI

**Backend Development & Risk
Assessment Module**

Kanuri Hema Sri

Team 1

Table of Contents

1. Introduction.....	3
2. Role Description	3
3. Objectives	3
4. Technologies Used	3
5. Environment Setup	4
6. Backend Project Structure.....	4
7. API Development	5
8. Request & Response Validation	6
9. Risk Assessment Module	6
10. API Testing	6
11. Future Work.....	7
12. Conclusion	7

1. Introduction

MedAssist AI is an AI-powered healthcare assistance system designed to support preliminary disease prediction and health risk assessment. The system aims to collect patient information, process symptoms, assess health risks, and provide predictions that can assist healthcare professionals.

2. Role Description

My responsibility in this project is to develop the FastAPI backend for the MedAssist AI project. The backend provides REST APIs for communication between the frontend and the machine learning model. It validates patient information, performs risk assessment, and returns structured responses. During this milestone, I completed the backend setup, API development, request and response validation, and the initial implementation of the Risk Assessment module.

3. Objectives

- Set up the FastAPI backend.
- Create a professional backend project structure.
- Develop REST API endpoints.
- Implement request and response validation.
- Build the Risk Assessment API.
- Prepare the Predict API for model integration.
- Test APIs using Swagger UI.
- Prepare the backend for future integration.

4. Technologies Used

Technology	Purpose
Python	Primary programming language used for backend development.
FastAPI	Backend framework used to build RESTful APIs.
Uvicorn	ASGI server used to run the FastAPI application.
Pydantic	Used for request and response validation.
Swagger UI	Automatically generated API documentation and testing interface.
Visual Studio Code	Integrated Development Environment (IDE) used for coding and debugging.
Git	Version control system used to manage project changes.
Virtual Environment (venv)	Used to isolate project dependencies and maintain a consistent development environment.

5. Environment Setup

The backend development environment was configured using Python and FastAPI. After completing the setup, the backend server was successfully launched using Uvicorn and verified through Swagger UI.

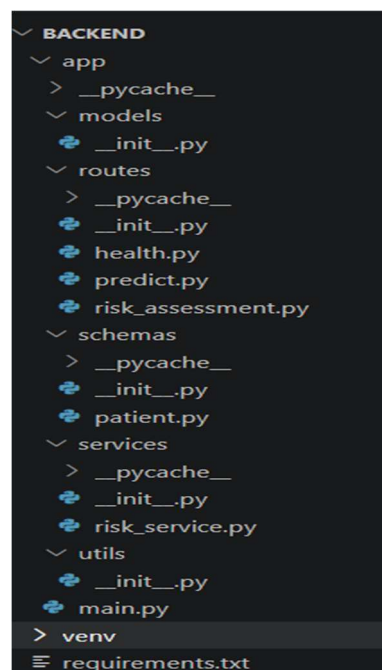
The following setup was completed:

- Installed Python
- Created a Virtual Environment (venv)
- Installed FastAPI
- Installed Uvicorn
- Installed Pydantic
- Generated requirements.txt
- Configured the backend project

6. Backend Project Structure

A modular backend project structure was designed using FastAPI to improve code organization and maintainability. The project is divided into folders such as routes, schemas, services, models, and utils, where each folder has a specific responsibility. This structure makes the application easier to develop, test, and maintain. It also prepares the backend for future integration with the machine learning model and frontend.

Structure:



7. API Development

Three REST APIs were developed to support backend communication and health risk assessment.

GET /health

Purpose

Checks whether the backend server is running successfully.

Expected Response

```
{  
  "status": "OK",  
  "message": "MedAssist AI Backend is running"  
}
```

POST /predict

Purpose

Receives patient information and prepares the backend for disease prediction model integration.

At present, the endpoint returns a placeholder response because the machine learning model is still under development.

Expected Response

```
{  
  "predicted_disease": "Model not integrated yet",  
  "confidence": 0.0  
}
```

POST /risk-assessment

Purpose

Receives patient information and evaluates the patient's health risk.

The endpoint returns one of the following risk levels:

- High Risk
- Medium Risk
- Low Risk

Expected Response

```
{
```

```
"risk_level": "Medium"
}
```

8. Request & Response Validation

Request and response validation was implemented using Pydantic.

Patient Requests

The backend validates:

- Age
- Gender
- Symptoms

Response Models

The backend returns structured responses for:

- Disease Prediction
- Risk Assessment

Validation ensures that incorrect or incomplete data is rejected before processing.

9. Risk Assessment Module

An initial rule-based Risk Assessment module was implemented. The current logic is based on patient symptoms. The current implementation serves as an initial version and can be extended with more advanced medical rules or machine learning models in future milestones.

Symptoms	Risk Level
Chest Pain	High
Difficulty Breathing	High
Fever + Cough	Medium
Other Symptoms	Low

The logic is implemented inside the Services layer, allowing future improvements without changing the API endpoints.

10. API Testing

All developed APIs were tested using FastAPI Swagger UI.

The following endpoints were successfully verified:

- GET /health
- POST /predict

- POST /risk-assessment

The API responses were validated to ensure correct functionality.

11. Future Work

The following tasks will be completed in the next milestone:

- Integrate the trained disease prediction model.
- Improve the risk assessment logic.
- Connect the backend with the frontend.
- Perform end-to-end system testing.

12. Conclusion

During Milestone 1, the backend foundation for MedAssist AI was successfully developed using FastAPI. The project includes a modular backend architecture, REST APIs, request and response validation, and an initial rule-based risk assessment module. All APIs were tested successfully using Swagger UI. The backend is now ready for integration with the disease prediction model and frontend in the next milestone.